

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

ABSTRACT FACTORY REPORT

CS202

Group members:

1. 25125030 - Võ Gia Minh
2. 25125055 - Phạm Huy Khánh
3. 25125066 - Nguyễn Quốc Thịnh
4. 25125085 - Chu Nguyễn Gia Khánh

21st August 2026



Contents

1	A Real-World Problem	2
2	A Naive Solution	2
3	Problems With The Naive Solution	3
3.1	Violation of Open-Closed Principle	3
3.2	Tight Coupling	4
3.3	Family Consistency Is Not Guaranteed	4
3.4	Duplicate Creation Logic	4
3.5	Maintenance And Class Explosion In Conditions	5
3.5.1	Maintenance	5
3.5.2	Class Explosion In Conditions	6
4	An Introduction About Your Pattern	6
5	Implementation	7
5.1	The general class diagram for the abstract factory design pattern	7
5.2	The abstract factory implementation of the problem raised at the beginning section	8
6	Pros and Cons of the Abstract Factory Pattern	8
6.1	Advantages	8
6.2	Disadvantages	9
7	Real-World Applications in Web, Mobile, and Software Development	9
7.1	Database Connectivity and Data Access Layers	10
7.2	Game Development and Level Theming	10
7.3	Document Export Systems	11
8	Quizzes	11
9	References	15
10	Appendices	15

1 A Real-World Problem

A software company is developing a cross-platform text editor that is expected to run on multiple operating systems, including Windows, macOS, and Linux. To provide users with a native experience, the application must display user interface components such as buttons, checkboxes, text fields, and menus that match the visual style and behavior of each operating system. As the project grows, the company also plans to support additional platforms and introduce new interface components while maintaining a consistent look and feel across each operating system. The challenge is to design the application in a way that can accommodate different families of user interface components while remaining flexible enough to support future expansion and changing requirements.

2 A Naive Solution

A straightforward approach to solving this problem is to let the application create the required user interface components directly by instantiating their corresponding concrete classes. The client code determines the target operating system at runtime and uses conditional statements, such as `if` or `switch`, to decide which objects to create. For example, if the application is running on Windows, it creates instances of `WindowsButton`, `WindowsCheckbox`, and `WindowsTextField`; if it is running on macOS, it creates `MacButton`, `MacCheckbox`, and `MacTextField` instead. This process is repeated wherever user interface components are needed, allowing the application to display platform-specific elements based on the selected operating system.

For the naive solution, we may write code like this:

```
1    if (os == "Windows") {
2        button = new WindowsButton();
3        checkbox = new WindowsCheckbox();
4    }
5    else if (os == "Mac") {
6        button = new MacButton();
7        checkbox = new MacCheckbox();
8    }
```

3 Problems With The Naive Solution

Although the naive approach is simple and works well in the early stages of development, it becomes less effective as the application grows. As new platforms and features are introduced, the code gradually becomes more difficult to maintain, extend, and manage. These challenges suggest that a more flexible and scalable approach is needed to support future changes without increasing the complexity of the system.

3.1 Violation of Open-Closed Principle

The naive approach also violates the ****Open/Closed Principle****, which states that software entities should be open for extension but closed for modification. Whenever a new product family or platform is added, existing client code must be modified to accommodate the new concrete classes. As a result, extending the system requires altering previously tested code, increasing the risk of introducing bugs and making the application more difficult to maintain.

Let consider an application that supports two operating systems: Windows and macOS. The client code uses conditional statements to determine which button to create:

```
1   Button* button;  
2   if (os == "Windows") {  
3       button = new WindowsButton();  
4   } else if (os == "macOS") {  
5       button = new MacButton();  
6   }
```

Suppose the application is later extended to support Linux. To accommodate the new platform, the existing client code must be modified:

```
1   Button* button;  
2   if (os == "Windows") {  
3       button = new WindowsButton();  
4   } else if (os == "macOS") {  
5       button = new MacButton();  
6   } else if (os == "Linux") {  
7       button = new LinuxButton();  
8   }
```

Although the application has only been extended with a new platform, the original client code had to be changed. This violates the Open/Closed Principle because the existing implementation is not closed for modification. As the number of supported platforms continues to grow, more changes to the client code will be required, making the system increasingly difficult to maintain and extend.

3.2 Tight Coupling

The naive approach also results in **tight coupling** between the client and the concrete product classes. Since the client directly creates objects such as ‘WindowsButton’ or ‘MacButton’, it becomes dependent on their specific implementations rather than their abstract interfaces. Consequently, any changes to the concrete classes or the introduction of new product families may require modifications to the client code. This strong dependency reduces the flexibility of the system, making it more difficult to maintain, extend, and reuse in different contexts.

3.3 Family Consistency Is Not Guaranteed

Another limitation of the naive approach is that it does not guarantee the consistency of product families. Because the client is responsible for creating each object individually, it is possible to accidentally combine products from different families, such as using a Windows button together with a macOS checkbox. This can lead to an inconsistent user interface and unexpected behavior, especially in applications where related components are designed to work together. As the number of product families increases, maintaining consistency becomes increasingly difficult and error-prone.

3.4 Duplicate Creation Logic

Another drawback of the naive approach is the duplication of object creation logic throughout the application. Since each screen or module is responsible for creating its own user interface components, the same conditional statements for selecting platform-specific objects are often repeated in multiple places, such as the **login screen**, **settings screen**, **main menu**, and **inventory screen**. While this may seem manageable initially, any change in requirements—such as supporting a new operating system or updating an existing UI theme—requires developers to modify the creation logic in every affected location.

For example, you have these screens:

```
1   LoginScreen
2   SettingsScreen
3   MainMenu
4   InventoryScreen
```

Every screen repeats the following code:

```
1   if(os == "Windows")
2   {
3       ...
4   }
5   else if(os == "Mac")
6   {
7       ...
8   }
```

All duplicate the same creation code, then requirements change:

```
1   Windows 11 Theme
2   Dark Theme
3   Linux Theme
```

This repetitive process is not only time-consuming but also increases the likelihood of inconsistencies, maintenance overhead, and implementation errors, making the application more difficult to evolve over time.

3.5 Maintenance And Class Explosion In Conditions

3.5.1 Maintenance

Another limitation of the naive approach is that it reduces the testability of the application. Since the client directly instantiates concrete classes such as `WindowsButton`, it becomes impossible to substitute these objects with alternative implementations during testing. For instance, instead of using a real `WindowsButton`, developers may wish to inject a `TestButton` or mock object to verify the client's logic without relying on platform-specific behavior. However, because the client is responsible for creating the object itself, such substitution is difficult to achieve without modifying the production code. This tight dependency complicates unit testing, increases the effort required to write and maintain tests, and makes the codebase less flexible for future development.

3.5.2 Class Explosion In Conditions

As the application evolves, the naive approach can also lead to **class explosion in conditional statements**. Each time a new product family or platform is introduced, additional conditions must be added to determine which concrete classes should be instantiated. When multiple product types—such as buttons, checkboxes, text fields, and menus—are involved, these conditional statements become increasingly large and are often repeated throughout the application. Over time, the growing number of concrete classes and corresponding conditions makes the code more complex, harder to read, and more difficult to maintain. This increasing complexity also raises the likelihood of implementation errors whenever new product families or components are added.

4 An Introduction About Your Pattern

The Abstract Factory is a creational design pattern that allows you to produce families of related or dependent objects without specifying their concrete classes. Instead of creating individual objects using separate factory methods or direct instantiations, the pattern provides an interface for creating an entire suite of matching products. This ensures that the products created together are always compatible with one another.

In a naive design, client code might manually instantiate each component of a complex UI or system—such as combining a `WinButton` with a `MacCheckbox`. If the creation logic is scattered across conditional statements (`if-else`), adding a new family of products (like Linux UI controls) requires modifying every place where objects are created. Abstract Factory solves this by centralizing the creation of related product families into dedicated factory classes, ensuring client code remains decoupled from concrete product implementations.

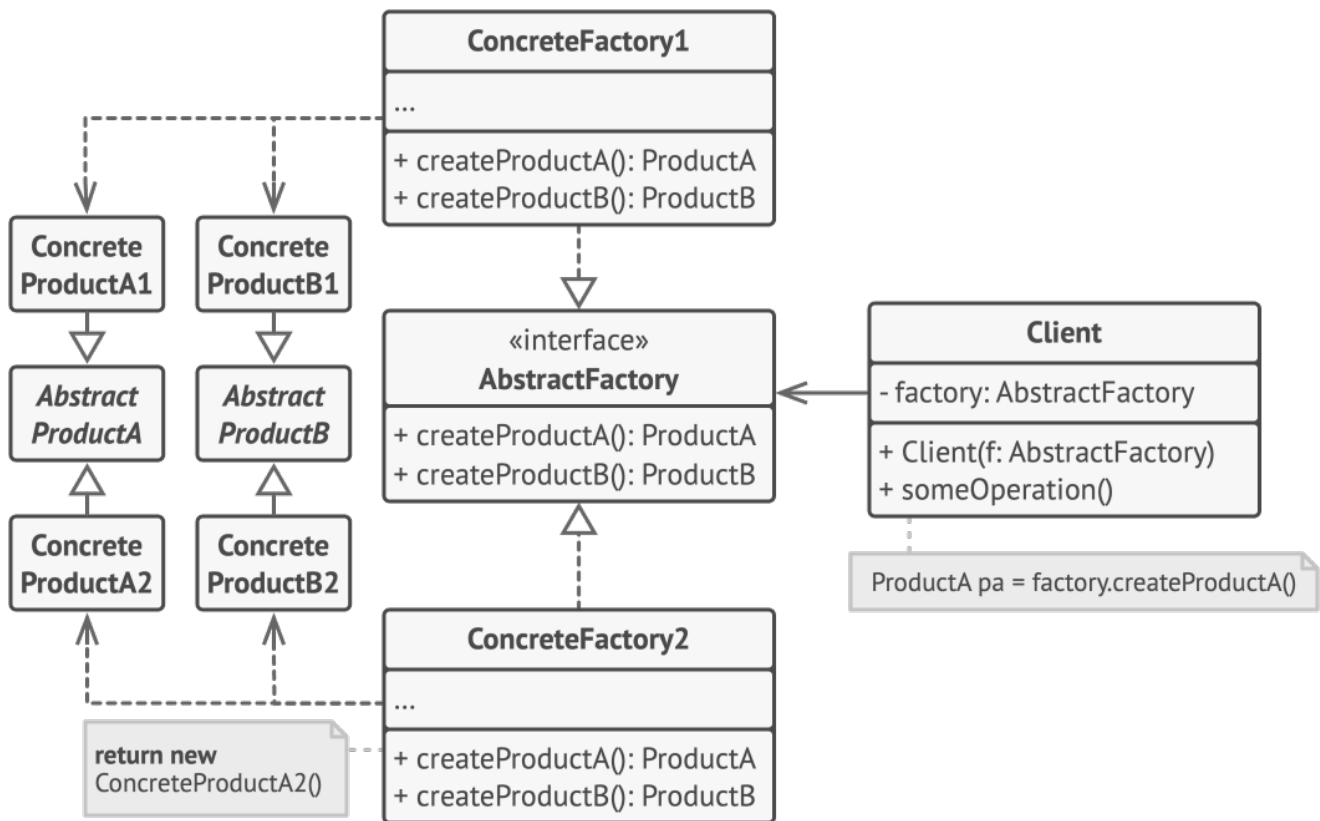
The Abstract Factory pattern contains five main roles:

- **AbstractProduct**: An interface or abstract class for a distinct type of product within a family. For example, `Button` or `Checkbox`.
- **ConcreteProduct**: Specific implementations of abstract products, grouped by variants. For example, `WinButton` and `MacButton` both implement `Button`, while `WinCheckbox` and `MacCheckbox` implement `Checkbox`.

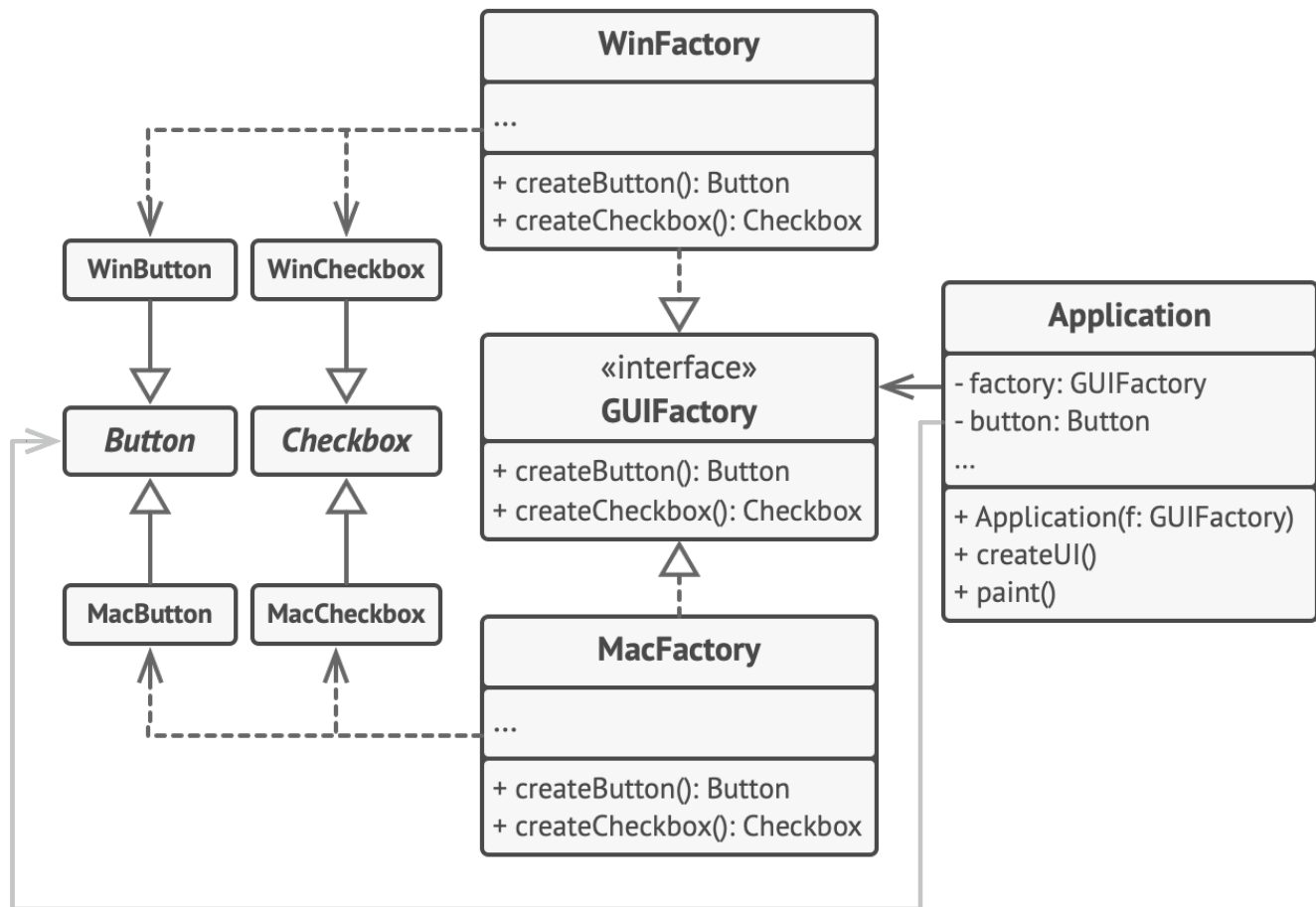
- **AbstractFactory**: An interface declaring a set of creation methods for each abstract product type in the family (e.g., `createButton()` and `createCheckbox()`).
- **ConcreteFactory**: Specific classes that implement the **AbstractFactory** creation methods to produce a concrete variant of the product family. For example, **WinFactory** creates **WinButton** and **WinCheckbox**, while **MacFactory** creates **MacButton** and **MacCheckbox**.
- **Client**: The application code that uses only the interfaces declared by **AbstractFactory** and **AbstractProduct**. It works with any concrete factory and product variant seamlessly without tight coupling.

5 Implementation

5.1 The general class diagram for the abstract factory design pattern



5.2 The abstract factory implementation of the problem raised at the beginning section



Click here to see the source code for the problem.

6 Pros and Cons of the Abstract Factory Pattern

The Abstract Factory Pattern provides several benefits, especially for systems that need to support multiple families of related objects and future extensions. However, it also introduces additional structure, so it should be used only when the added flexibility is worth the complexity.

6.1 Advantages

- **Promotes the Dependency Inversion Principle:** The pattern ensures that client code depends on high-level abstractions, such as interfaces, rather than concrete implementations.

- **Decouples the Client from Implementations:** By returning abstract product types rather than concrete classes, the pattern isolates the client from implementation details.
- **Ensures Product Consistency:** The pattern is explicitly designed to create families of related or dependent objects, ensuring that a client always receives compatible objects from the same family.
- **Ease of Adding Product Families:** While adding new product types is challenging, adding entirely new product families, such as a new operating-system GUI factory, is straightforward and does not require changes to existing client code.

6.2 Disadvantages

- **Difficulty Adding New Product Types:** Supporting new kinds of products, such as adding a `createScrollbar()` method, is highly rigid. It requires altering the main Abstract Factory interface and cascading those changes across all existing concrete factory implementations.
- **Increased System Complexity:** The pattern introduces many new interfaces and classes into the codebase. For smaller systems or simpler creation logic, this overhead can overcomplicate the architecture.
- **Upfront Design Overhead:** The pattern requires correctly predicting the variations of product families early in the design phase. If the system's needs are not well understood, the rigid structure of the factory interfaces can become a hindrance.

7 Real-World Applications in Web, Mobile, and Software Development

The Abstract Factory Pattern is a creational design pattern used to produce families of related or dependent objects without specifying their concrete classes. It is particularly useful when a system must support multiple themes, environments, platforms, or output formats while ensuring that the objects created for each variant remain compatible with one another.

The following examples demonstrate common real-world applications of the Abstract Factory Pattern.

7.1 Database Connectivity and Data Access Layers

Enterprise applications are frequently designed to support several database management systems, such as SQL Server, Oracle, PostgreSQL, and MySQL. Although these systems provide similar operations, their concrete connection, command, and data-reader objects are not interchangeable.

- **Abstract Factory:** `DbProviderFactory`
- **Abstract Products:** `DbConnection`, `DbCommand`, and `DbDataReader`
- **Concrete Factories:** `SqlClientFactory`, `OracleClientFactory`, and other provider-specific factories
- **How it works:** The application selects a factory based on its database configuration. All connections, commands, and readers are then created through that provider's factory.
- **Benefit:** The pattern guarantees that all data-access objects belong to the same provider family. For example, an Oracle command cannot accidentally be used with a SQL Server connection.

7.2 Game Development and Level Theming

Games commonly contain levels or maps with distinct themes, such as fantasy, science fiction, or post-apocalyptic environments. Although each level requires enemies, weapons, and obstacles, the concrete objects should match the active theme.

- **Abstract Factory:** `LevelElementFactory`
- **Abstract Products:** `Enemy`, `Weapon`, and `Obstacle`
- **Concrete Factories:** `FantasyFactory` and `SciFiFactory`
- **How it works:** When the player enters a fantasy level, the game uses `FantasyFactory` to create related objects such as an `Orc`, a `Sword`, and a fantasy-themed obstacle. Switching to `SciFiFactory` allows the same spawning logic to create an `Alien`, a `LaserBlaster`, and science-fiction obstacles.
- **Benefit:** The game engine can change an entire family of level elements without modifying its spawning workflow, while ensuring that all generated entities remain thematically consistent.

7.3 Document Export Systems

Reporting applications often allow users to export the same information to formats such as PDF, Word, and HTML. The logical structure of a report remains similar, but each format requires different rendering instructions and concrete elements.

- **Abstract Factory:** `DocumentExporterFactory`
- **Abstract Products:** `Header`, `Paragraph`, and `Table`
- **Concrete Factories:** `PdfExporterFactory`, `HtmlExporterFactory`, and `WordExporterFactory`
- **How it works:** The report-generation engine processes the same source data and asks the selected factory to create headers, paragraphs, and tables. If the user chooses PDF export, every element is created using PDF-compatible rendering instructions; selecting HTML produces corresponding HTML elements instead.
- **Benefit:** Export formats can be selected or extended without coupling the report-generation workflow to format-specific classes, and all elements within an exported document remain compatible with the selected format.

8 Quizzes

Question 1: What is the primary purpose of the Abstract Factory Design Pattern?

- A. To ensure a class only has one instance and provide a global point of access to it.
- B. To provide an interface for creating families of related or dependent objects without specifying their concrete classes.
- C. To separate the construction of a complex object from its representation.
- D. To allow an object to alter its behavior when its internal state changes.

Answer: B

Explanation: Abstract Factory provides an interface for creating entire families of compatible

products while keeping their concrete classes hidden from the client.

Question 2: How does the Abstract Factory Pattern differ from the Factory Method Pattern?

- A. Factory Method creates families of objects, while Abstract Factory creates a single object type.
- B. Abstract Factory relies on inheritance, while Factory Method relies entirely on object composition.
- C. Abstract Factory creates families of related objects, whereas Factory Method is typically responsible for creating one specific type of product.
- D. There is no difference; they are different names for the same pattern.

Answer: C

Explanation: Abstract Factory coordinates the creation of multiple related product types. Factory Method usually defines a method for creating one product type, often through subclassing.

Question 3: Which SOLID principle does the Abstract Factory Pattern most heavily promote by ensuring clients depend on interfaces rather than concrete implementations?

- A. Single Responsibility Principle
- B. Open/Closed Principle
- C. Liskov Substitution Principle
- D. Dependency Inversion Principle

Answer: D

Explanation: The Dependency Inversion Principle encourages clients to depend on abstractions. Abstract Factory allows clients to use abstract factory and product interfaces instead of concrete classes.

Question 4: What is a common disadvantage of using the Abstract Factory Pattern?

- A. It couples client code tightly to concrete classes.
- B. Supporting new kinds of products, such as adding a `createScrollbar()` method, is difficult because it requires changing the Abstract Factory interface and all of its concrete implementations.
- C. It makes adding new product families, such as a Linux GUI factory, extremely difficult.
- D. It exposes the instantiation logic to the client.

Answer: B

Explanation: Adding a new product type changes the abstract factory contract, so every concrete factory must be updated to create that product.

Question 5: Which of the following real-world scenarios is the best fit for implementing an Abstract Factory?

- A. A cross-platform UI toolkit that needs to render Windows-style and macOS-style buttons, checkboxes, and text fields.
- B. A logging system that writes to either a file or a database, depending on configuration.
- C. A document parser that builds a complex DOM tree step-by-step.
- D. A caching system where objects are cloned to avoid expensive database calls.

Answer: A

Explanation: A cross-platform UI toolkit must create families of visually and behaviorally compatible controls for each operating system, making it a classic Abstract Factory use case.

Question 6: In the Abstract Factory Pattern, which component is ultimately responsible for instantiating the concrete objects?

- A. The Client
- B. The Abstract Factory Interface

C. The Concrete Factories

D. The Abstract Products

Answer: C

Explanation: Concrete factories contain the creation logic and instantiate the concrete products belonging to a particular product family.

Question 7: Which other design pattern is frequently used to implement a Concrete Factory, since typically only one instance of a specific factory is needed?

A. Observer

B. Singleton

C. Strategy

D. Decorator

Answer: B

Explanation: The Singleton Pattern can ensure that only one shared instance of a concrete factory is responsible for creating its product family.

Question 8: If an `AbstractFactory` interface has methods such as `createButton()` and `createCheckbox()`, what should be the return types of these methods?

A. Concrete classes, such as `WindowsButton` and `MacCheckbox`

B. Abstract product types, such as `IButton` and `ICheckbox`

C. Generic object types, such as `Object` and `Any`

D. Boolean values indicating whether creation was successful

Answer: B

Explanation: Returning abstract product types keeps the client independent of concrete implementations and allows an entire product family to be replaced by selecting a different concrete factory.

9 References

1. Refactoring.Guru, “Abstract Factory,” *Design Patterns*. Available: <https://refactoring.guru/design-patterns/abstract-factory>.
2. SourceMaking, “Abstract Factory Design Pattern,” *Design Patterns*. Available: https://sourcemaking.com/design_patterns/abstract_factory.
3. TutorialsPoint, “Abstract Factory Pattern,” *Design Patterns*. Available: https://www.tutorialspoint.com/design_pattern/abstract_factory_pattern.htm.
4. GeeksforGeeks, “Abstract Factory Design Pattern,” *Software Design Patterns*. Available: <https://www.geeksforgeeks.org/abstract-factory-pattern/>.

10 Appendices